

视觉扩散模型中的稀疏混合专家路由： 从路由坍塌到选择性锁死的诊断、边界标定与进化路线图

Haiying Sha*

Independent Researcher

May 12, 2026

Abstract

混合专家模型 (Mixture-of-Experts, MoE) 已成为语言建模领域扩展模型容量的核心范式, 但在视频生成领域, 扩散模型 Diffusion Transformer (DiT) 上的应用仍处于起步阶段, 现有尝试的收益十分有限。本文系统性地诊断了 Token-Choice 稀疏 MoE 在视频扩散模型中的训练动力学失效模式。我们从预训练的 Dense 视频生成模型 (约 50 亿参数) 出发, 将其转换为 MoE 架构: 路由专家直接从原始 Dense 的 FFN 权重精确复制, 共享专家采用极小的非零噪声初始化 (如 10^{-4} 量级, 验证阶段则采用全零以保证输出等价), 仅路由器从随机初始化开始学习。

实验揭示了一系列关键的层次化失效: (1) 线性路由器经历全局软饱和, Softmax 梯度消失导致专家完全同质化 (余弦相似度 >0.99); (2) 非线性 MLP 路由器解决了全局锁死, 但引入更隐蔽的选择性锁死, 约三分之一的层退化为单专家模式, 且增大辅助损失至 0.2 也无法阻止; (3) 利用多模态特征与视频特征进行交叉注意力融合的路由器, 展现出初步的自我恢复能力——这是所有架构中唯一观察到的锁死层主动回弹现象, 但最终仍有约 9 层顽固锁死; (4) 这些锁死层的分布呈现出显著的 U 型结构: 高度集中于浅层视觉处理层 (第 1–10 层) 和深层语义整合层 (第 18–25 层), 而中间层保持接近均匀的利用率; (5) bfloat16 混合精度导致小量级专家权重更新被硬件截断为零——一个极易被忽视的数值陷阱。

通过追踪 30 层 Transformer 中超过 6500 万 token 在 5000 步训练中的路由决策时间序列, 我们建立了完整的锁死现象学。我们将此现象解释为“门控-共享专家-路由专家”三元系统中的功能冗余动态, 提出“功能冗余假说”——该假说预测, 全零 / 极微初始化且尚未充分训练的共享专家必须达到一个临界能力阈值, 系统才能释放被沉默的路由专家。这一假说与系统生物学中的功能冗余理论高度一致, 为理解 MoE 训练动力学提供了全新的理论视角。

在工程层面, 我们记录了 Dense-to-MoE 转换的三大法则 (专家结构一致性、1:1 权重复制、共享专家验证全零与训练微噪声初始化), 以及 bfloat16 精度陷阱的完整解决方案。在此基础上, 我们进一步探索了多模态特征序列拼接范式 (In-Context Conditioning)、基于 MoE 的文字渲染解决方案 (引入专用文字专家 + Glyph-ByT5 字形编码器), 以及音视频联合生成的进化路径。

基于上述诊断发现, 我们规划了从“视觉统一”到“通用智能引擎”的三步进化路线图: 近期通过 MoE 化实现定向增强, 中期通过增加音频专家实现跨模态音视频联合生成, 远期融入物理先验和因果推理能力, 构建世界模型并赋能具身智能。

核心论点: 在当前的 Token-Choice 范式下, 部分层的锁死是结构性的, 与路由器的复杂度无关; 要彻底消除视觉 MoE 中的死专家问题, 需要从共享专家的初始化策略、专家权重的差异化设计、或路由范式本身的升级入手。本文代表了针对现实视频生成 MoE 部署场景中路由失效的首次全面经验性诊断之一, 为后续研究提供了明确指引。

▷ GitHub <https://github.com/Shybert-AI/UniGen-MOE>

1 引言

1.1 背景与动机

混合专家模型 (Mixture-of-Experts, MoE) (1; 2) 已成为扩展深度学习模型容量的核心范式。其基本思想——使用稀疏门控机制将每个输入 token 动态路由至一组专门的“专家”子网络——在语言建模领域取得了非凡成功。DeepSeek-V4 (1.6 万亿参数, 384 专家/层, 每 token 激活 6 个) (3)、Qwen3 系列 (4)、以及 Kimi K2.6 等系统均采用 MoE 架构, 在保持计算效率的同时实现了参数数量和性能的数量级跃升。这些系统的成功建立在专家专业化的关键假设之上: 不同的子网络学会处理输入空间中不重叠、语义上有意义的区域。

*Corresponding author: Yan Zheng.

然而，将 MoE 应用于视觉生成——特别是视频 Diffusion Transformer (DiT)——的探索仍处于初期阶段。尽管 ProMoE(5)、DiffMoE(6) 和 Mamoda2.5(7) 等工作已将 MoE 模块引入扩散 Transformer，但缺乏完整的失效诊断。我们认为，视觉 MoE 和语言 MoE 之间存在根本性区别：语言 token 在语义上高度浓缩且离散，而视觉 patch token 展现出极端的空间冗余性 (5; 8)、由扩散时间步引起的分布漂移 (9)，以及在多任务设置下路由信号来源的冲突 (10)。这些差异使得语言领域成熟的路由机制在视觉生成上面临系统性未知风险。

1.2 研究问题与主要贡献

本文旨在系统性地诊断和标定 Token-Choice 稀疏 MoE 在视频扩散 Transformer 上部署时的失效模式。我们从预训练的 Kiwi-Edit 视频编辑模型出发，该模型由 Wan2.2-TI2V-5B 视频扩散主干和 Qwen2.5-VL-3B-Instruct 多模态编码器（总计约 80 亿参数）组成。我们使用三大法则将其转换为 MoE：保留专家结构一致性、1:1 权重复制路由专家、以及共享专家在验证阶段采用全零而训练时启用微小噪声（如 10^{-4} ）的初始化策略。后我们让路由器从零开始学习专家分工。实验设置的关键特征——极低噪声的共享专家、两个权重复制的路由专家、9 合 1 (17) 多任务视频生成场景——创造了一个极端严苛的试炼，任何路由失效都无法被预训练能力所掩盖。

我们的主要贡献如下：

1. 首次全面诊断：提供了 Token-Choice 稀疏 MoE 在视频扩散模型中的失效模式系统分类，涵盖线性路由器到交叉注意力路由器的完整架构演进对比，建立了完整的锁死现象学。
2. 发现“选择性锁死”现象：识别并命名了一种独特的失效模式，它在消除全局锁死后出现，由门控、共享专家和路由专家之间的三元互动驱动。该现象表现为约 1/3 的层退化为单专家使用模式，且增大辅助损失无法阻止。
3. 发现“U 型锁死分布”：通过 30 层网络、超过 6500 万 token 的时序数据，首次精确描绘了锁死层的结构化分布——高度集中于浅层视觉处理层和深层语义整合层，中间层保持均衡。
4. 提出“功能冗余假说”：基于 5000 步、超 34 亿 token 的路由决策时序数据，将锁死重新解释为一种理性的、阶段性的资源分配策略。该假说从系统生物学中寻找理论支撑，为理解 MoE 训练动力学提供了全新的理论视角。
5. 工程实践贡献：记录了 Dense-to-MoE 转换的三大法则、bfloat16 混合精度训练下 MoE 专家权重更新的精度陷阱及其完整解决方案，并明确了共享专家验证全零与训练微噪的初始化范式。
6. 标定 Token-Choice 范式的能力边界：通过实验数据精确标定了当前范式的训练边界，为后续向 Expert-Choice 或动态路由机制的研究提供基准与指引。
7. 进化路线图：基于诊断发现，规划了从多模态统一到音视频联合生成，最终通往世界模型的三步进化路径。

1.3 论文结构

本文按以下结构组织：第 2 节回顾相关工作；第 3 节介绍 Dense-to-MoE 转换的三大法则和三种路由器架构；第 4 节详细展开五个核心实验及其诊断结果；第 5 节提出功能冗余假说并讨论其理论；第 6 节标定 Token-Choice 范式的能力边界；第 7 节探讨多模态统一框架的序列拼接范式；第 8 节分析文字渲染的 MoE 解决方案；第 9 节规划从视觉统一到世界模型的进化路线图；第 10 节总结全文并展望未来工作。

2 相关工作

2.1 语言模型中的混合专家模型

混合专家模型的概念可追溯至 Shazeer 等人 (1) 的开创性工作，他们通过引入稀疏门控机制，使模型容量得以呈数十亿参数量级扩展。此后，MoE 迅速成为大语言模型扩展的事实标准。DeepSeek 系列 (3) 将 MoE 推向极致——其 V4 模型 (2026 年) 拥有 1.6 万亿参数，每层配备 384 个专家，每个 token 激活 6 个，总激活参数仅为 49B。这些模型在 SWE-bench Verified、LiveCodeBench 等基准上的出色表现，确立了 MoE 在语言模型中的统治地位。

然而，语言 MoE 的成功依赖于一个关键假设的有效性：输入 token 具有高度的语义不连续性。每个文本 token 代表一个相对独立的信息单位，token 间差异显著。这使得路由器能够依赖隐藏状态的几何结构自然涌现专家分化 (11)，而无需显式的专业化引导。正如 Dai 等人 (11) 的近期实证研究

所揭示的, MoE 中观察到的专家专业化模式更可能由隐藏状态的几何结构自然涌现, 而非路由架构的主动诱导。

2.2 视觉生成中的混合专家模型

将 MoE 应用于视觉生成领域的尝试仍处于早期阶段。Wei 等人 (5) 在 ICLR 2026 上发表的 ProMoE 框架, 首次系统性分析了视觉 MoE 与语言 MoE 的关键差异: 视觉 token 存在高度空间冗余性和功能异质性(源于无分类器引导机制), 导致隐式路由策略产生次优结果。ProMoE 提出两步路由框架——条件路由和原型路由——以促进专家专业化, 在 ImageNet 基准上超越了现有方法。

Shi 等人 (7) 的 Mamoda2.5 将 DiT-MoE 设计(128 个专家, Top-8 路由)引入统一多模态框架, 在视频编辑质量上创造了新纪录。Mamoda2.5 采用 Sigmoid 门控与 loss-free Expert Bias 负载均衡策略, 并提出了基于随机神经元采样的 Upcycling 初始化方法, 加速了 MoE 的收敛。该工作在工业界验证了视觉 MoE 在大规模部署下的可行性, 但其对路由行为的深层机理分析相对有限。

DREAM 框架 (10) 通过时间-上下文感知门控实现动态专家路由, 展示了扩散时间步嵌入作为路由条件信息的潜力。Jiang 等人 (8) 提出的 Mixture of Distributions 为高效视频扩散 Transformer 引入了动态稀疏注意力机制。在工业应用方面, 快手的 DiffMoE(6) 在 CVPR 2025 上展示了扩散模型中 MoE 架构的初步成果。

尽管这些工作取得了显著进展, 但它们往往聚焦于架构创新和性能提升, 对训练过程中路由行为的动态演变、尤其是失效模式的系统诊断关注有限。本研究试图填补这一空白: 在一个极简却严苛的实验设置下(共享专家极低噪声初始化、权重复制路由由专家、多任务混合训练), 完整记录从路由坍塌到选择性锁死的全过程。

2.3 负载均衡与路由坍塌

负载均衡是 MoE 训练中的经典挑战。标准方法包括辅助损失 (1) 和专家容量限制 (2)。最近, 基于相似度保持的路由器 (14) 被证明能加速收敛并降低冗余。然而, 这些方法的共同前提是路由信号的梯度仍然有效——一旦进入 Softmax 饱和区, 所有惩罚机制的梯度趋于零, 调节信号无法传递。在我们的实验中, 将辅助损失系数从 0.01 提升至 0.08 对锁死层的专家分布几乎不产生影响(变化量 < 0.05%), 印证了这一局限。该观察也与 Dai 等人 (11) 的发现一致: MoE 中的路由行为更多由数据几何结构决定, 而非简单的惩罚机制所能改变。

2.4 混合精度训练的数值陷阱

bfloat16 已成为大模型训练的默认精度, 其 8 位指数与 float32 相同, 但 7 位尾数限制了其最小可分辨步长(Unit in the Last Place, ULP)。在 MoE 训练中, 当可训练参数的量级极小(如全零或极微初始化的共享专家)或极大(如 Dense 权重复制的路由专家)时, 优化器的单步更新量可能低于 bfloat16 在该数值区间的 ULP, 导致权重更新被硬件截断为零。

我们在第 4.4 节将详细展示这一现象的具体表现及其解决方案, 该方案的核心在于让可训练参数的主副本保持 float32, 仅在前向计算时通过 autocast 自动转换为 bfloat16——这是对工业界混合精度训练最佳实践的独立验证。

2.5 路由机制的范式演进

现有的 MoE 路由机制主要分为三类: Token-Choice 路由 (1; 2) (每个 token 选择 top-k 专家)、Expert-Choice 路由 (15) (每个专家选择 top-k token) 以及动态路由(如 DREAM(10) 的时间-上下文感知门控)。Token-Choice 已被广泛证明在语言模型中有效, 但其在视觉扩散模型中的适应性问题尚未得到充分探索。本文聚焦于 Token-Choice 范式, 通过对路由器架构的系统性对比(线性、MLP、交叉注意力), 帮助理解路由复杂度与失效模式之间的关系。

2.6 多模态特征融合的序列拼接范式

在视频生成领域, 多模态特征融合采用序列拼接(Sequence Concatenation)的方式已成为主流范式。SkyReels-V4 采用对称双流多模态扩散 Transformer (MMDiT) 架构, Mamoda2.5 在 DiT-MoE 主干中使用 In-Context Conditioning 注入多模态条件。这些模型的共同特征是将文本、图像、音频等不同模态的特征在序列维度拼接后送入统一的 Transformer 主干进行处理。本文在第 7 节将探讨这一范式在 MoE 框架下的独特优势。

3 Dense-to-MoE 转换:三大法则与路由器架构

3.1 转换的本质与总体框架

我们从预训练的 Kiwi-Edit 视频编辑模型出发,该模型由 Wan2.2-TI2V-5B 视频扩散主干和 Qwen2.5-VL-3B-Instruct 多模态编码器(总计约 80 亿参数)组成。该模型是一个 Diffusion Transformer (DiT),包含 30 层 Transformer block,每层 FFN 的中间维度为 14336,隐藏维度为 3072。我们将其转换为 MoE 架构,每层配备 2 个路由专家和 1 个共享专家。

MoE 转换的本质是将原本单一的前馈网络 (FFN) 拆分为多个独立、可被动态路由的“专家”子网络:

$$\text{FFN}(x) = W_2 \cdot \text{GELU}(W_1 \cdot x + b_1) + b_2$$

↓ MoE 化

$$\text{MoE}(x) = \sum_{i=1}^{N_s} \text{FFN}_i^{(s)}(x) + \sum_{j=1}^{N_r} g_j(x) \cdot \text{FFN}_j^{(r)}(x)$$

其中 $\text{FFN}_i^{(s)}$ 为共享专家 (所有 token 可见), $\text{FFN}_j^{(r)}$ 为路由专家 (由 Gate 动态选择), $g_j(x)$ 为 Gate 输出的路由权重, N_s 和 N_r 分别为共享专家和路由专家的数量。

3.2 三大法则之一:专家结构一致性

法则陈述:路由专家必须与原始 FFN 的结构完全一致——使用相同的 GELU 激活、相同的 Linear 层数量和相同的 bias 配置。任何结构偏差都会导致权重映射失败。

坑点详述:我最早写的 ExpertMLP,是参照 DiT-MoE (18) 官方代码写的 SwiGLU 结构:

错误示范: SwiGLU 结构

```
class ExpertMLP(nn.Module):
    def __init__(self, hidden_size, intermediate_size):
        self.gate_proj = nn.Linear(hidden_size, intermediate_size, bias=False)
        self.up_proj = nn.Linear(hidden_size, intermediate_size, bias=False)
        self.down_proj = nn.Linear(intermediate_size, hidden_size, bias=False)

    def forward(self, x):
        return self.down_proj(self.act(self.gate_proj(x)) * self.up_proj(x))
```

这是 SwiGLU——用两个线性层分别算“门”和“值”,再相乘。但我的原始模型 FFN 是最朴素的 GELU MLP:

$$\text{FFN}(x) = \text{fc2}(\text{GELU}(\text{fc1}(x)))$$

结构和参数形状都对不上。复制过去的权重张量形状不匹配,模型第一层就崩了。

之所以犯错,是因为我当时混淆了两种在视觉模型中常见的 FFN (前馈网络) 变体。一些开源 MoE 教程采用的是 SwiGLU 激活,而我需要转换的基线模型是基于标准 GELU 激活的 MLP。由于二者在参数量、权重形状和门控机制上都存在根本性差异,直接套用 SwiGLU 结构的代码无法兼容原有权重,导致模型在第一轮推理时即告崩溃。

解决方案:把专家改成和原始 FFN 一样的 $\text{fc1} \rightarrow \text{GELU} \rightarrow \text{fc2}$,并且保留 bias (因为原始模型有 bias)。

3.3 三大法则之二：路由专家 1:1 复制，不做缩放

法则陈述：每个路由专家的权重从原始 Dense 的 FFN 权重精确复制。不进行任何缩放。

坑点详述：结构对上之后，我以为大功告成。跑推理——输出一片空白，没有任何有效信号。我开始怀疑是数值幅度的问题，于是加了一个缩放因子。

在查阅相关文献时，我发现针对 MoE 专家输出的缩放策略存在两种不同的理论依据：一种基于方差保持原则，主张使用 $1/\sqrt{N}$ 缩放（Gaussian 初始化下的统计不变性）；另一种基于和的守恒原则，主张使用 $1/N$ 缩放（直接控制输出幅值）。我逐一进行了验证：用 $1/3$ （2 个路由专家 + 1 个共享专家），结果崩溃；换成 $1/2$ （按期望激活数），同样崩溃。

经过深入分析后，我发现：这些根号缩放、方差缩放策略，全部是为“从零开始随机初始化训练一个新的 MoE”设计的。在该场景下，专家权重初始为随机分布，不同专家的输出相互独立，需要通过缩放因子来保证总和的方差与原始 FFN 保持一致。这是标准的统计初始化理论。

在随机初始化 + 从零训练的场景下，权重刚开始是随机的，不同专家的输出会相互独立。为了让总和的方差和原始 FFN 保持一致，需要用 $1/\sqrt{N}$ 或类似因子来控制。这是统计学原理。

但我的场景完全不同。我拿的是一个已经训练好的 Dense 模型，每一层的归一化参数、残差连接的比例、甚至解码器对输出尺度的预期，全部是基于“这一层的输出大约是 $1.0 \times f(x)$ ”这个假设训练的。

你一把砍成 0.577 或 0.5，信号逐层衰减，经过几十层网络之后，输出分布完全偏离了预期区间。解码一跑，直接崩溃。

关键洞察：这个坑的根本原因是把“从零训练”的初始化策略，套用在了“已训练模型转换”的场景上。两个问题看起来相似，但解法完全不同。

解决方案：路由专家的缩放因子设为 1.0，不做任何缩放。

3.4 三大法则之三：共享专家初始化策略——验证全零，训练微噪

法则陈述：共享专家在架构验证阶段置于全零以保持与原始 Dense 的输出等价；实际训练时则采用极小的非零噪声（如标准差 10^{-4} ）初始化，以打破对称性并避免 bfloat16 下的更新停滞。

坑点详述：几乎每一篇讲 MoE 架构的文章，都说“共享专家用于学习通用知识”——但没说该怎么初始化。最初我自然地想到全零：它能让 MoE 在转换后的推理输出与原 Dense 完全一致，从数学上保证起点正确。然而在实际训练中，当共享专家权重严格为零时，其梯度信号极其微弱，尤其在 bfloat16 混合精度下，优化器单步更新量往往低于硬件可分辨的最小步长，导致参数完全无法被“唤醒”。这一问题将在第 4.4 节中详细剖析。

经过反复实验，我最终确定了一套分阶段的初始化方案：

1. 验证阶段（全零）：将共享专家权重全部置零。推理时共享专家输出为零，MoE 输出 $= \sum g_j \cdot \text{FFN}_j^{(r)}$ 。当路由专家权重复制且 Gate 未训练时，输出与原始 Dense 模型严格一致。这用于校验架构实现的正确性。
2. 训练阶段（微噪）：实际启动训练前，用极小的噪声（如 $\mathcal{N}(0, (10^{-4})^2)$ ）重新初始化共享专家权重，偏置保持零。这个微小的扰动几乎不改变推理输出（误差低于 10^{-6} 量级），但足以在 bfloat16 下产生可积累的梯度，使共享专家能够逐步学习通用知识。

代码示例：

```
# 验证用全零
shared_w0 = torch.zeros(shared_dim, inner_dim)
shared_w2 = torch.zeros(inner_dim, shared_dim)
shared_b0 = torch.zeros(shared_dim)
shared_b2 = torch.zeros(inner_dim)

# 训练用微小噪声（替换上面）
shared_w0.normal_(mean=0, std=1e-4)
shared_w2.normal_(mean=0, std=1e-4)
# bias 保持零或同样微小噪声均可
```

这样做的逻辑是：

- 推理时(验证阶段): 共享专家输出为零, MoE 输出完全等于原始 Dense 模型。
- 训练时: 共享专家从一个极低但非零的起点出发, 立即获得有效梯度, 逐渐学习通用知识。Gate 也在训练中分化, 逐渐为不同 token 选择不同专家。

它同时保证了推理起点的数值正确性和训练阶段的优化可行性。

3.5 推理验证: 输出完全一致

用全零共享专家生成权重后, 我跑了同一个推理命令。结果和原始 Dense 模型完全一样——不是模糊相近, 是数值级一致(因为共享专家为零时, 数学上两者等价)。

这不是失败, 这是验证。它证明:

- MoE 架构(Gate、dispatch、组合逻辑)没有 bug;
- 权重映射完全正确;
- 模型可以正常启动训练。

验证准则: 转换后的 MoE, 在验证阶段应该和原 Dense 模型输出完全一致。如果不一致, 说明初始化策略或架构实现有问题。实际训练时再替换为微噪初始化。

3.6 三大法则总结

三个不要	三个要
不要把从零训练 MoE 的缩放公式用在 DenseToMoE 转换上	路由专家要 1:1 复制原始权重, 不做任何缩放
不要改动专家的网络结构	共享专家验证用全零, 训练用微小噪声
不要一开始就解冻所有参数	训练启动时要解冻整个 MoE 模块

3.7 路由器架构

我们实现了三种不同的路由器进行对比实验:

3.7.1 线性路由器 (Linear Gate)

最基本的 Linear(3072, 2) $\rightarrow$ Softmax 结构。两个 logits 通过全连接层投影后再通过 Softmax 归一化, 每个 token 独立选择得分最高的专家。这是语言 MoE 中最常见的路由形式。

```
class MoEGate(nn.Module):
    def __init__(self, embed_dim, num_experts=16, num_experts_per_tok=2,
                  aux_loss_alpha=0.01, layer_idx: Optional[int] = None,
                  gate_init_std: float = 0.01):
        super().__init__()
        self.top_k = num_experts_per_tok
        self.n_routed_experts = num_experts
        self.layer_idx = layer_idx
        self.scoring_func = 'softmax'
        self.alpha = aux_loss_alpha
        self.seq_aux = False
        self.norm_topk_prob = False
        self.gating_dim = embed_dim
        self.weight = nn.Parameter(torch.empty((self.n_routed_experts, self.gating_dim)))
        nn.init.normal_(self.weight, mean=0.0, std=gate_init_std)
    def forward(self, hidden_states: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor, Optional[torch.Tensor]]:
        bsz, seq_len, h = hidden_states.shape
        hidden_states = hidden_states.view(-1, h)
        logits = F.linear(hidden_states, self.weight, None)
```

```

scores = logits.softmax(dim=-1)
topk_weight, topk_idx = torch.topk(scores, k=self.top_k, dim=-1, sorted=False)
if self.top_k > 1 and self.norm_topk_prob:
    denominator = topk_weight.sum(dim=-1, keepdim=True) + 1e-20
    topk_weight = topk_weight / denominator
aux_loss = None
if self.training and self.alpha > 0.0:
    scores_for_aux = scores
    topk_idx_for_aux = topk_idx.view(bsz, -1)
    if self.seq_aux:
        ce = torch.zeros(bsz, self.n_routed_experts, device=hidden_states.device)
        ce.scatter_add_(1, topk_idx_for_aux,
                        torch.ones(bsz, seq_len * self.top_k, device=hidden_states.device)
                        ).div_(seq_len * self.top_k / self.n_routed_experts)
        aux_loss = (ce * scores_for_aux.view(bsz, seq_len, -1).mean(dim=1)).sum(dim=1).mean()
    else:
        mask_ce = F.one_hot(topk_idx_for_aux.view(-1), num_classes=self.n_routed_experts)
        ce = mask_ce.float().mean(0)
        Pi = scores_for_aux.mean(0)
        fi = ce * self.n_routed_experts
        aux_loss = (Pi * fi).sum() * self.alpha
if self.training and MoEGate._log_enabled and self.layer_idx is not None:
    with torch.no_grad():
        batch_experts = topk_idx.cpu().view(-1).tolist()
        for e in batch_experts:
            MoEGate._log_buffer[self.layer_idx][e] += 1
        MoEGate._log_step_count[self.layer_idx] += len(batch_experts)
return topk_idx, topk_weight, aux_loss

```

3.7.2 MLP 路由器 (MLP Gate)

Linear(3072, 256) → GELU → Linear(256, 2) → Softmax。引入非线性决策边界以增强表达能力。最后一层使用微小随机噪声 (标准差 0.002) 初始化, 以避免全零初始化导致的 topk 对称选择问题——当所有 logits 相等时, topk 默认返回第一个索引, 导致全局锁死在专家 0。

```

class MLPGate(nn.Module):
    def __init__(self, embed_dim, num_experts=16, num_experts_per_tok=2,
                  aux_loss_alpha=0.01, layer_idx: Optional[int] = None,
                  gate_init_std: float = 0.002):
        super().__init__()
        self.top_k = num_experts_per_tok
        self.n_routed_experts = num_experts
        self.layer_idx = layer_idx
        self.scoring_func = 'softmax'
        self.alpha = aux_loss_alpha
        self.seq_aux = False
        self.norm_topk_prob = False
        self.gating_dim = embed_dim

        hidden_dim = 256
        self.gate_mlp = nn.Sequential(
            nn.Linear(embed_dim, hidden_dim),
            nn.GELU(),
            nn.Linear(hidden_dim, num_experts)
        )
        # 最后一层权重用微小噪声初始化, 偏置置零
        nn.init.normal_(self.gate_mlp[2].weight, mean=0.0, std=gate_init_std)
        nn.init.zeros_(self.gate_mlp[2].bias)

```

```

def forward(self, hidden_states: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor, Optional[torch.Tensor]]:
    bsz, seq_len, h = hidden_states.shape
    hidden_states = hidden_states.view(-1, h)
    logits = self.gate_mlp(hidden_states) # [bsz*seq_len, num_experts]
    scores = logits.softmax(dim=-1)
    topk_weight, topk_idx = torch.topk(scores, k=self.top_k, dim=-1, sorted=False)
    if self.top_k > 1 and self.norm_topk_prob:
        denominator = topk_weight.sum(dim=-1, keepdim=True) + 1e-20
        topk_weight = topk_weight / denominator

    aux_loss = None
    if self.training and self.alpha > 0.0:
        scores_for_aux = scores
        topk_idx_for_aux = topk_idx.view(bsz, -1)
        if self.seq_aux:
            ce = torch.zeros(bsz, self.n_routed_experts, device=hidden_states.device)
            ce.scatter_add_(1, topk_idx_for_aux,
                           torch.ones(bsz, seq_len * self.top_k, device=hidden_states.device)
                           ).div_(seq_len * self.top_k / self.n_routed_experts)
            aux_loss = (ce * scores_for_aux.view(bsz, seq_len, -1).mean(dim=1)).sum(dim=1).mean()
        else:
            mask_ce = F.one_hot(topk_idx_for_aux.view(-1), num_classes=self.n_routed_experts)
            ce = mask_ce.float().mean(0)
            Pi = scores_for_aux.mean(0)
            fi = ce * self.n_routed_experts
            aux_loss = (Pi * fi).sum() * self.alpha

    if self.training and MoEGate._log_enabled and self.layer_idx is not None:
        with torch.no_grad():
            batch_experts = topk_idx.cpu().view(-1).tolist()
            for e in batch_experts:
                MoEGate._log_buffer[self.layer_idx][e] += 1
            MoEGate._log_step_count[self.layer_idx] += len(batch_experts)

    return topk_idx, topk_weight, aux_loss

```

3.7.3 交叉注意力路由器 (Cross-Attention Gate)

利用底层 QwenVL-3B 的多模态编码能力。一个轻量的 `nn.MultiheadAttention` (2 个头) 让每个图像 token 作为 Query 检索文本指令的 Key/Value, 然后通过线性头投影出专家 logits。这使得路由决策能够利用文本指令的语义信息, 实现任务感知的灵活路由。该模块将总参数量从 10.31B 增加到 11.21B (新增约 0.9B 参数)。

```

class MoEGate(nn.Module):
    def __init__(self, embed_dim, num_experts=16, num_experts_per_tok=2,
                 aux_loss_alpha=0.01, layer_idx: Optional[int] = None,
                 gate_init_std: float = 0.01, encoder_dim: int = 3072):
        super().__init__()
        self.top_k = num_experts_per_tok
        self.n_routed_experts = num_experts
        self.layer_idx = layer_idx
        self.scoring_func = 'softmax'
        self.alpha = aux_loss_alpha
        self.seq_aux = False
        self.norm_topk_prob = False
        self.gating_dim = embed_dim

        # 轻量级交叉注意力路由
        self.router_attn = nn.MultiheadAttention(

```

```

        embed_dim=embed_dim,
        num_heads=2,          # 极简头数
        kdim=encoder_dim,     # Key 来自文本编码器
        vdim=encoder_dim,     # Value 来自文本编码器
        batch_first=True
    )
    # 最终投影到 num_experts 个专家
    self.router_head = nn.Linear(embed_dim, num_experts)
    # 关键: router_head 全零初始化, 保证训练起点绝对均匀
    nn.init.normal_(self.router_head.weight, std=0.002)
    nn.init.zeros_(self.router_head.bias)
    # router_attn 内部参数保持 PyTorch 默认随机初始化 (打破对称性)

def forward(self, hidden_states: torch.Tensor,
            encoder_hidden_states: Optional[torch.Tensor] = None) -> Tuple[torch.Tensor, torch.Tensor]
    bsz, seq_len, h = hidden_states.shape

    # 如果没有文本特征, 退化为简单的线性路由 (兼容性保护)
    if encoder_hidden_states is None:
        # Fallback: 用 hidden_states 自身做自注意力路由
        attn_out, _ = self.router_attn(
            query=hidden_states,
            key=hidden_states,
            value=hidden_states
        )
    else:
        # 交叉注意力: 图像 token 查询文本信息
        attn_out, _ = self.router_attn(
            query=hidden_states,
            key=encoder_hidden_states,
            value=encoder_hidden_states
        )

    logits = self.router_head(attn_out) # [B, seq_len, num_experts]

    # 以下与原 Gate 完全一致
    scores = logits.softmax(dim=-1)
    topk_weight, topk_idx = torch.topk(scores, k=self.top_k, dim=-1, sorted=False)
    if self.top_k > 1 and self.norm_topk_prob:
        denominator = topk_weight.sum(dim=-1, keepdim=True) + 1e-20
        topk_weight = topk_weight / denominator

    aux_loss = None
    if self.training and self.alpha > 0.0:
        scores_for_aux = scores
        topk_idx_for_aux = topk_idx.view(bsz, -1)
        if self.seq_aux:
            ce = torch.zeros(bsz, self.n_routed_experts, device=hidden_states.device)
            ce.scatter_add_(1, topk_idx_for_aux,
                           torch.ones(bsz, seq_len * self.top_k, device=hidden_states.device)
                           ).div_(seq_len * self.top_k / self.n_routed_experts)
            aux_loss = (ce * scores_for_aux.view(bsz, seq_len, -1).mean(dim=1)).sum(dim=1).mean()
        else:
            mask_ce = F.one_hot(topk_idx_for_aux.view(-1), num_classes=self.n_routed_experts)
            ce = mask_ce.float().mean(0)
            Pi = scores_for_aux.mean(0)
            fi = ce * self.n_routed_experts
            aux_loss = (Pi * fi).sum() * self.alpha

```

```

        if self.training and MoEGate._log_enabled and self.layer_idx is not None:
            with torch.no_grad():
                batch_experts = topk_idx.cpu().view(-1).tolist()
                for e in batch_experts:
                    MoEGate._log_buffer[self.layer_idx][e] += 1
                MoEGate._log_step_count[self.layer_idx] += len(batch_experts)

    return topk_idx, topk_weight, aux_loss

```

3.8 专家利用率监控

我们实现了细粒度的专家利用率日志系统，每 50 个训练步记录一次每个 MoE 层的专家激活统计。对于每层的 2 个路由专家，我们计算：

- 少数专家占比 (minority expert fraction): 被分配给冷门专家的 token 比例，值域为 $[0, 0.5]$ 。当该值低于 10% 时，我们认为该层进入“锁死”状态。
- 利用率标准差 (std_utilization): 衡量两个专家之间分配均衡度的指标。当标准差接近 0 时，表示均衡分配；接近 0.5 时，表示极端偏向某一专家。
- 平均利用率: 在多个日志记录间隔内的平均统计，用于评估长期趋势。

这些指标使我们能够追踪路由决策在时间上的动态演变，并在早期检测到锁死信号。

3.9 训练设置

实验在 9 合 1 任务统一视频生成数据集上进行，涵盖文生图 (text-to-image)、图生视频 (image-to-video)、视频编辑 (video-editing)、图像编辑 (Image-editing)、文生视频 (text-to-video) 等 9 类任务，共 20,000 个训练样本。

训练配置如下：

- batch_size = 1
- 视频帧数 = 9
- 最大像素数 = 345,600
- 学习率 = $2e-4$ (线性 warmup 500 步后余弦衰减)
- 优化器 = 8-bit AdamW (betas=[0.9, 0.999], eps= $1e-8$)
- 精度 = bfloat16 混合精度 (可训练参数主副本保持 float32)
- 可训练参数 = 门控网络 + 共享专家 (路由专家冻结)
- 硬件 = NVIDIA A100-SXM4-80GB X 1
- 总训练步数 = 5,000 步

4 实验与诊断: 五种路由失效模式的完整病理学

4.1 实验一: 线性路由器的全局软饱和

设置: 采用线性路由器 (Linear $\rightarrow$ Softmax)，共享专家以微小噪声初始化 ($\sigma = 1e-4$)，路由专家权重复制，aux_loss_alpha = 0.01。

观察 (2000 步): 利用率数据显示约 26 层看起来“健康” (少数专家占比超过 20%)，仅 4 层 (第 1、4、20、27 层) 表现出严重偏向。然而，这一表面健康的数据具有欺骗性。进一步诊断揭示：

- 路由专家输出完全同质化: 随机抽取同一批 token 输入后的两个专家输出进行比较，余弦相似度超过 0.99。表明两个专家学到了几乎完全相同的特征，MoE 退化成了“结构不均匀的 Dense 网络”——路由器的存在只引入了额外计算开销，而没有带来任何能力增益。
- 共享专家成长极慢: 经过 2000 步训练后，共享专家权重范数仍然极低，尚未学到足够的通用知识。



- **aux_loss 无效**: 将 `aux_loss_alpha` 从 0.01 提高到 0.08, 对锁死层的少数专家占比没有产生任何可测量的影响(变化量 $<0.05\%$)。

诊断: 线性路由器的表达能力天生受限。它在高维潜空间中只能画一条“直线”(一个投影方向), 而视觉 token 的特征分布——尤其是在 9 合 1 任务混合下——根本无法被如此简单的决策边界有效分割。更关键的是, Softmax 在 logits 差距较大时进入饱和区, 梯度趋近于零, 使得任何基于梯度的负载均衡策略都失效了。这不是参数调优问题, 而是结构性的表达能力不足。

4.2 实验二: MLP 路由器的选择性锁死

设置: 将路由器升级为非线性 MLP (Linear $\rightarrow$ GELU $\rightarrow$ Linear $\rightarrow$ Softmax), 最后一层权重使用微小噪声初始化 (标准差 0.002), 偏置保持零。aux_loss_alpha 保持 0.01。共享专家仍以微小噪声起始。其余设置不变。

观察(初期): MLP 路由器确实解决了全局软饱和——训练初期(前 100 步), 所有 30 层的少数专家占比均超过 1%, 标准差的分布范围恢复至 0.1–0.5 的健康范围。这是整个实验的第一个里程碑: 非线性 MLP Gate + 微小噪声初始化的组合拳, 被证明能 100% 消除视觉 MoE 的全局路由锁死问题。

观察(后期): 随着训练从 1000 步推进至 5000 步, 一种更隐蔽的失效模式出现了: 选择性锁死。

Table 1: MLP 路由器在不同训练步数的关键层少数专家占比演变

层号	1000 步	2000 步	3000 步	5000 步	锁死趋势
第 1 层	36.35%	18.32%	12.21%	7.33%	深度锁死
第 4 层	—	40.20%	26.80%	16.08%	濒临锁死
第 6 层	—	41.95%	27.97%	16.78%	濒临锁死
第 10 层	—	36.88%	24.59%	14.75%	深度锁死
第 16 层	—	34.92%	23.28%	13.97%	深度锁死
第 19 层	—	36.50%	24.33%	16.03%	濒临锁死
第 21 层	—	37.41%	24.94%	15.07%	深度锁死

到 5000 步时, 第 1 层的少数专家占比降至 7.33%, 第 10 层降至 14.75%, 第 16 层降至 13.97%, 第 19 层降至 16.03%, 第 21 层降至 15.07%。同时, 第 3、7、9、25 层的主专家占比均超过 85%, 构成反向锁死。

诊断: MLP 的非线性赋予了路由器更强的表达能力, 但这种“果断”也让它学得更快、更极端。关键机制是门控-共享专家-路由专家三元互动: 随着共享专家逐渐学会通用知识 (weight_norm 从微量增长到 ~ 30), 门控发现“选择一个主路由专家 + 共享专家”已经足以处理大多数 token——另一个路由专家无法提供额外价值。被冷落的路由专家收不到梯度, 门控权重更新进一步固化偏好, 形成自激正反馈的死亡螺旋。

将 `aux_loss_alpha` 提高到 0.2 也未能阻止这一趋势: 在 softmax 饱和区, 梯度信号极其微弱, 单纯提高惩罚系数无济于事。这进一步证实了锁死是系统层面的结构性行为, 而非可单独通过负载均衡解决的局部问题。

4.3 实验三: bfloat16 精度陷阱

设置: 采用交叉注意力路由器。共享专家的初始化遵从前述策略: 验证阶段全零, 正式训练时使用极小噪声 ($\sigma = 1e-4$)。这一微小扰动确保了在 bfloat16 下能够被优化器识别, 否则全零初始化的共享专家在混合精度训练下可能完全无法更新。训练精度为 bfloat16。

观察: 训练开始后, 共享专家的梯度范数始终在 0.004–0.009 之间波动——反向传播是正常的。但若采用严格全零初始化, 共享专家的权重范数 (weight_norm) 连续数百步保持完全不变。打印单个权重值 (如 `fc1.weight[0, 0]`) 发现它精确地卡在 0.0000002384 或直接为零。

Table 2: bfloat16 精度陷阱的数值分析

参数	数值	说明
共享专家学习率	0.04	全局 $lr \times 20$
梯度范数 (典型)	~ 0.005	正常回传, 非零
单步理论更新量	$0.04 \times 0.005 = 0.0002$	微量
权重值范围	~ 115.5 (或 ~ 0.01)	取决于初始化
bfloat16 ULP @ 115.5	~ 0.5	最小可分辨步长
bfloat16 ULP @ 0.01	$\sim 7e-8$	极小值区域的 ULP

当权重值在 115.5 附近时, 更新量 0.0002 远小于最小可分辨步长 0.5, 导致 $w+ = lr * grad$ 的操作在硬件层面被直接舍入为零。而全零初始化下的权重为零, 更新量虽然可能落在 ULP 内, 但由于对称性和零点附近的舍入规则, 实际参数变化难以累积。

解决方案: 采用两个关键措施: (1) 共享专家训练时启用微小噪声初始化 ($\sigma = 1e-4$), 使其权重脱离绝对零点; (2) 让可训练参数的主副本保持 float32, 仅在前向计算时通过 `torch.autocast` 自动转换为 bfloat16。修改后, 共享专家权重范数立即从约 0.0115 开始稳步增长:

- 300 步时: 3.33
- 500 步时: 19.24
- 800 步时: 29.53

关键洞察: 这是混合精度训练中一个极易被忽视的陷阱——当可训练参数的量级与更新步长均极小时, bfloat16 的精度可能不足以记录参数的任何变化。该解决方案的核心在于参数主副本保持 float32, 同时共享专家从一个极小的非零噪声出发, 从而避免了全零初始化在 bfloat16 下的更新死锁。

4.4 实验四: 交叉注意力路由器的终局

设置: 采用交叉注意力路由器 (基于 QwenVL-3B 的多模态编码), 共享专家采用前述微噪初始化, 路由专家保持冻结。

观察 (自我恢复行为): 交叉注意力路由器在训练过程中表现出一种独特的主动回弹现象。在 warmup 阶段, 部分原本濒临锁死的层逐渐恢复活跃度。例如, 第 2 层的少数专家占比从极低水平 (约 0.11%) 逐步攀升至 4% 以上; 第 28 层则从 8% 持续上升至 18%, 成功跨越了 10% 的健康基准线。这是本系列实验中唯一观察到的路由器自我恢复现象。

交叉注意力路由器之所以具备这一特性, 源于其能够利用文本指令信号区分功能相似但语义各异的 token, 从而实现更灵活的路由分配。其 Gate 本质上是一个多模态感知器, 以图像特征为 Query, 主动“询问”文本指令中的语义信息。该设计使路由决策能够从丰富的多模态特征池中动态提取最相关的条件信号, 有效缓解了单模态路由器的路径依赖问题。

观察 (锁死层分布): 尽管出现了自我恢复迹象, 但随着训练推进至中后期, 顽固锁死层的总体数量并未显著减少——仍有约 9 层处于锁死或严重偏斜状态。这些锁死层的分布呈现出显著的非随机结构化特征: 高度集中在第 2–8 层 (浅层视觉处理层) 和第 23–29 层 (深层语义整合层), 而健康层则大量分布于中间段 (第 9–22 层)。

Table 3: 交叉注意力路由器训练后期各层路由健康状态示例(数据取自 Step 3000/5000)

类型	层号	少数专家占比	状态
锁死层	第 1 层	7.33%	深度锁死
锁死层	第 4 层	26.80%	偏斜
锁死层	第 6 层	27.97%	偏斜
锁死层	第 10 层	24.59%	深度锁死
锁死层	第 16 层	23.28%	深度锁死
锁死层	第 19 层	24.33%	深度锁死
健康层	第 11 层	49.80%	均衡
健康层	第 12 层	49.99%	均衡
健康层	第 15 层	50.13%	均衡
健康层	第 20 层	49.94%	均衡
健康层	第 26 层	49.93%	均衡
健康层	第 27 层	49.96%	均衡

这种分布模式与 ProMoE(5) 的核心观察高度一致: 视觉 token 的功能异质性与空间冗余性对浅层路由构成了更大挑战。浅层主要处理原始低级视觉特征, token 间可区分性极弱, 极易导致路由坍塌。与此同时, 深层路由器在聚合全部中间层信息后, 决策空间被高度压缩, 一旦形成路径偏好便极难跳出局部最优。

关键对照实验: 在早期的一版实验中, 因代码配置失误, 共享专家被错误初始化为预训练 Dense 模型的完整权重(而非微噪初始化)。在该设置下, 路由利用率表现显著改善——锁死层比例降至 10% 以下, 远优于当前正式实验。

这一“意外”提供了重要的间接证据: 一旦共享专家具备成熟的通用表征能力(即使仅继承自预训练 Dense 权重), 路由器的专家坍塌/锁死现象即可得到显著缓解。相比之下, 在当前的实验设定下(共享专家仅含极微噪声、双路由专家权重复制), Gate 在训练初期缺乏任何先验知识基础, 必须完全从零构建专家分工。这构成了对路由器架构最严苛的试炼, 也进一步凸显了共享专家初始化策略在 MoE 训练中的关键作用。

4.5 实验五: 全专家训练的尝试与限制

设置: 基于交叉注意力路由器的最佳结果, 在单卡条件下尝试解冻路由专家以验证全专家协同训练的可行性。采用 8-bit AdamW 优化器, 解冻包括门控网络、共享专家和路由专家在内的全部 MoE 参数约 9.06B 参数。

观察: 全专家训练尝试立即遇到了显存瓶颈。将全部 9.06B 参数转换为 float32 主副本并进行混合精度训练时, 单卡显存峰值达到 79.22 GiB, 超出 A100 80GB 的实际可用上限。

Table 4: 全专家训练的显存占用分析

组件	bf16 参数	f32 参数	梯度 (f32)	优化器 (8-bit)	合计
路由专家 (2.64B)	5.28GB	10.56GB	10.56GB	5.28GB	~26.4GB
共享专家 (1.32B)	2.64GB	5.28GB	5.28GB	2.64GB	~13.2GB
Gate(1.12B)	2.24GB	4.48GB	4.48GB	2.24GB	~11.2GB
冻结组件	~20GB	~20GB	0	0	~20GB
激活	~10GB	~10GB	0	0	~10GB
总计					~80.8GB

通过冻结路由专家仅训练门控网络和共享专家(约 2.64B 可训练参数), 训练得以在约 65GB 显存占用下稳定运行。

诊断: 由于单卡显存的硬性限制, 我们未能完成全专家协同训练。这一经验明确表明, 在 Token-Choice 范式下, 要同时实现专家路由的平衡和专家能力的差异化, 至少需要双 GPU 的 ZeRO-3 分片或更先进的专家 offloading 策略。我们尝试了双 GPU 部署, 但遇到了容器环境下 NCCL 通信隔离的配置问题。

5 讨论:功能冗余假说

5.1 锁死作为战略储备

我们提出功能冗余假说来解释观察到的选择性锁死现象。这一假说的核心比喻是“战略储备与双人协作”,它精准地捕捉了当前 MoE 的训练动力学:

想象一个成熟的两人协作团队(路由专家,权重复制自 Dense 模型),和一个刚入职的学徒(共享专家,仅有极微噪声初始化)。团队的工作方式是“两人 + 学徒”并行处理同一个任务,最终成果是所有人的贡献之和。学徒虽然参与,但起初几乎没有任何有效产出。

- 初始状态:两位师傅能力完全相同(权重复制自 Dense 模型),学徒输出近似为零(微噪初始化)。由于学徒几乎没有贡献,团队的有效产出完全依赖两位师傅。此时,负责分配奖金(梯度)的调度员(门控网络)和整个团队,都处于一个能力高度冗余的状态。
- 早期阶段:自然分流与冷板凳在并行工作模式下,系统天然倾向于最小化冗余。两位师傅能力完全相同,意味着他们可以互相替代。调度员很快发现,对于大多数任务,只需激活一位师傅就够了,另一位师傅即使不参与,最终结果也几乎一样好。这不是懒惰,而是系统在约束下(参数更新是稀疏的)坍缩出的高效稳态:与其让两个一样的师傅重复劳动,不如把计算资源(梯度)集中给其中一位,让他学得更快。于是,一位师傅被频繁激活成为“主力”,另一位则被选择性忽略,成为“战略储备”。学徒则继续“旁听”学习。
- 关键阶段:学徒成长,打破平衡学徒(共享专家)通过持续“旁听”(接收反向传播的梯度),逐渐学会了处理通用、基础的任务。他的输出从几乎为零变为有意义的贡献。这意味着,即使主力师傅和另一位师傅都不参与,学徒也能搞定一部分基础工作。
- 唤醒时刻:从储备到专精学徒的成长为系统带来了新的可能性。调度员发现,之前必须由主力师傅处理的一些通用任务,现在可以放心地交给学徒独立负责。这使得主力师傅被“释放”,可以去处理更复杂的任务。同时,那位长期待机的“储备”师傅,其输出也终于变得有价值——他的能力不再与主力完全重叠,因为团队的整体输出结构已经因学徒的成长而改变。调度员开始重新分配流量,之前被锁死的路由专家(储备师傅)开始被唤醒,并可能走向与主力师傅不同的专精方向。

这个过程的本質是“并行冗余系统中,新组件成长如何驱动旧组件重新配置”的动态过程。共享专家(学徒)的成长改变了系统的整体能力结构,使得之前因“冗余”而被锁死的路由专家,获得了新的功能定位。三者(主力、储备、学徒)并行工作,相互成就,推动系统从“简单冗余”走向“高效协作”。这不是一个简单的“调参能调好”的问题,而是一个需要时间的、分阶段的系统演化过程。

5.2 功能冗余:为直觉寻找理论框架

在系统生物学中,功能冗余(functional redundancy)是指多个物种或基因具有相似的功能,当环境变化导致某些功能单元失效时,其他具有相同功能的单元可以接管其职责,从而维持整个系统的稳定性。

在 MoE 的语境下,两个初始相同的路由专家构成了“路由冗余”,而共享专家和路由专家的协同构成了“能力冗余”。训练早期的锁死现象,可以被理解为一个冗余系统的自组织过程:

- 在核心功能(正确生成输出)的约束下,系统优先激活冗余最低、能力最强的单元(单个路由专家)。
- 随着共享专家逐渐学会通用知识(产生新的冗余),系统才具备释放其他路由专家进行专精化分工的条件。

那些被我们标记为“坏死”的层,从功能冗余理论的视角看,实际上是被系统战略性储备的冗余能力。它们不是“死”了,而是被暂时抑制了——它们的命运,取决于共享专家何时能提供足够的冗余支持。

这个理论解释让我对所有数据有了全新的理解:第一版实验(共享专家非零)路由那么好,是因为冗余一开始就存在;当前版交叉注意力 Gate 的锁死层多,是因为冗余还在建设中。第 8 层在 1650 步的短暂激活——少数专家利用率从 0 跳变到 1.4% 而后迅速归零——正是功能冗余正在积累的第一个震荡信号。

5.3 涌现的临界点

与一位在推理模型涌现领域有深入研究的学者交流后,我意识到我观察到的现象本质上就是一种涌现——只不过,DeepSeek 等推理模型在强化学习过程中涌现出的是新的推理能力(如思维链、自我反思),而我的 MoE 试图涌现出的是分工能力。

但这两种涌现共享一个核心逻辑:当一个系统由多个部件构成,且部件之间存在复杂的相互作用时,整个系统会突然表现出一种全新的、无法从单一部件直接预测的整体行为。它符合涌现的经典特征——存在一个隐藏的临界点,在这个点之前系统的行为变化极其缓慢,一旦越过临界点,一个新能力会突然出现甚至迅速成熟。

而我的 MoE 系统,可能也正站在这样一个临界点的前方。那个等待中的“涌现”,不是共享专家自身能力的成长,而是整个 MoE 分工结构的自发形成——当共享专家积累足够知识后,大量的层可能突然苏醒,专家间开始出现清晰的任务分工,就像一个生态系统在积累了足够的物种多样性后,突然形成一个稳定的食物网。

到底有没有这个临界点?如果有,它在哪里——是 2000 步、3000 步,还是 10000 步?当前没有人知道答案。但如果我们能在实验数据中观测到它,这将对 MoE 训练动力学最宝贵的系统性贡献。

5.4 功能冗余假说与其他理论的对比

Table 5: 功能冗余假说与相关理论的对比

理论	核心解释	与功能冗余假说的关系
MoE 专家专业化迷思 (11)	路由反映几何结构而非专业化	解释了为什么锁死层分布呈现结构性特征
ProMoE 显式路由引导 (5)	条件路由 + 原型路由促进专业化	验证了更强的路由信号可以缓解锁死
功能冗余假说 (本工作)	锁死是三元系统在共享专家成熟前的理性等待策略	为上述现象提供了统一的动力学理论框架

6 Token-Choice 范式的标定边界

通过从线性路由器到 MLP 路由器再到交叉注意力路由器的三部曲实验,我们在极端严苛的条件下(共享专家微噪初始化、路由专家权重复制、9 合 1 多任务混合训练),完成了对 Token-Choice 范式路由能力的最大可能性标定。

在当前实验条件下,我们得出一个明确的结论:

在单 GPU 部署、路由专家冻结的前提下,Token-Choice 范式无法实现 30 层全层的健康均衡分工。部分层的锁死是结构性的,与路由器的复杂度无关。

这一边界并非一个令人沮丧的终点。恰恰相反,它是一个被实验数据精确标定的科学边界:对于未来的研究者,如果要彻底消除视觉 MoE 中的死专家问题,我们建议从三个方向入手:

1. 改进共享专家的初始化策略:赋予其一定的预训练通用知识(如从预训练 Dense 模型继承初始权重),使其在训练初期就具备功能冗余。这可以大幅缓解训练早期的锁死。
2. 引入专家权重的差异化设计:如专家对比学习损失 (5),使专家从初始化阶段就具有明确的互补能力。
3. 从 Token-Choice 范式迁移到 Expert-Choice(15) 或动态路由范式:使专家拥有主动选择 token 的能力,打破被动接受分配的困境。

7 架构进化:多模态统一框架的序列拼接范式

7.1 加法 vs 拼接:为什么加法会失败?

在最初的 UniGen 实验中,我曾尝试将 MLLM 特征和 T5 特征进行加权融合,以增强模型的语义理解能力。结果令人沮丧:推理输出全黑。当时我以为是模型结构不支持,因此搁置了这个方向。

后来我才意识到，失败的原因与模型结构无关，而在于特征注入方式的根本性错误：

- 加法是“破坏性注入”：直接将两个编码器的特征相加，等同于在潜空间中强行混入未经对齐的信号。一旦权重设置不当，原有的 latent 分布被破坏，导致“黑图”或“白图”。
- 拼接是“非破坏性注入”：在序列维度拼接并不会修改原始特征本身，而是为模型提供一个更大的信息池。交叉注意力机制是选择性关注，而非强制融合。

7.2 序列拼接: In-Context Conditioning

当我把特征融合方式从“加法”改为“序列拼接”后，一切豁然开朗：

正确做法：序列拼接

```
unified_condition = torch.cat([text_features, glyph_features, audio_features], dim=1)
```

这种方式在工业界被称为 In-Context Conditioning，是 Seedance 2.0、SkyReels-V4、Mamoda2.5 等最前沿模型的标准做法。它的核心思想是：不修改任何模型结构，只修改输入序列的长度和内容，让 Transformer 的自注意力机制自动学习如何从丰富的多模态信息池中提取所需信号。

7.3 MoE 的独特优势：避免多模态特征污染

在 Dense 模型中，序列拼接的 unified_condition 被送入交叉注意力层后，所有的 token 都会从同一个大融合池中提取信息。虽然这行之有效，但共享的 FFN 在后续处理时，仍需“消化”来自所有模态的信息——这本身就存在任务冲突的风险。

而 MoE 架构，恰好在这一点上提供了更优雅的解释：

- 交叉注意力层（共享）：让所有 token 平等地访问多模态信息池。
- MoE 层（动态路由）：让不同类型的 token 被分配给不同的专家进行处理。文字区域的 token 被分配给文字专家，背景区域的 token 被分配给视觉专家，音频相关的 token 则被分配给音频专家。无关的特征根本不会被注入到对应的专家输出中。

这种“深度融合 + 精准分流”的机制，是 Dense 模型无法做到的。

8 文字渲染：从“崩坏”到“专家”的解决之路

8.1 问题诊断：为什么文字会崩坏？

在 UniGen 的早期测试中，我遇到了一个几乎所有扩散模型都会面临的经典难题：图片中的文字生成会崩坏。具体表现为：指定的文字无法正确拼写、字符之间错位、笔画模糊成一团。

这是扩散模型普遍面临的问题，并非仅仅源于训练数据不足，其本质是模型在字符层面的感知能力缺失：

- CLIP 是“散光”的语义构图师：其训练目标是图文宏观语义对齐，缺乏对字符精确形状的敏感性。
- MLLM 是“远视”的理解大师：Qwen2.5-VL-3B 虽然具备强大的视觉-语言理解能力，但它并非专为字形级别的精细控制而设计。
- T5 是“深度近视”的语言专家：通用 T5 能很好地理解句法和语意，但它并未专门对齐字符形状。直接用未经微调的 T5 特征去生成文字，会因为特征分布不匹配而导致崩坏。

8.2 技术路线抉择：T5 vs ByT5

要解决文字渲染，关键是让模型具备“字符感知”能力。当我在 T5 和 ByT5 之间抉择时，看似微小的差异实际上指向了完全不同的技术路径：

- ByT5：直接在原始 UTF-8 字节上操作，天然理解字符形状。它是解决文字渲染问题的最直接有效的基础模型。
- T5：虽然被广泛用于 Imagen、PixArt- α 等扩散模型，但它缺乏精确的字形对齐能力，更适合作为通用语义编码器。

结论: 只用一个, 选择 ByT5。T5-XXL 高达 110 亿参数(蒸馏后仅需 T5-base 即可达到相似的生成质量), 而 ByT5-Small 仅约 3 亿参数。对于一个独立研究者而言, 选择轻量级的 ByT5 不仅是资源上的务实选择, 更是功能上的正确选择。

8.3 MoE 的终极方案: 增加“文字专家”

然而, 仅仅引入 ByT5 还不够。这是扩散模型普遍面临的挑战, 并非简单地“换个编码器”就能根除, 它需要一种能够从根本上突破信息瓶颈的全新架构支撑。而 MoE 框架, 恰恰为此提供了完美的土壤。

研究界的先进方案早已指明了方向: 需要引入专门的 Glyph-ByT5 编码器, 通过“字形-文本”配对数据集微调, 让 ByT5 不仅理解字符, 更能精确感知字体的形状、笔画、字距和颜色。

在 MoE 框架下, 我无需修改任何现有组件, 只需:

1. 增加一个“文字专家”: 一个专门负责处理字符渲染任务的 FFN 子网络。
2. 利用 ByT5 的字形特征: 作为 Gate 路由决策的辅助条件。
3. 让 Gate 自动学习分工: 一段时间后, Gate 会发现“文字区域应该被分配给文字专家”。对于不包含文字的 token, Gate 分配给文字专家的权重为 0, 这个专家的输出完全不会被加到最终 latent 中。

这正是 MoE 架构的核心优势: 稀疏激活保证了计算效率, 动态路由实现了任务解耦。

9 进化路线图: 从视觉统一到通用智能引擎

基于上述诊断发现, 我们规划了从“视觉统一”到“通用智能引擎”的三步进化路线。

9.1 近期: 定向增强与 MoE 化

当前最迫切的任务, 是补上那些已被精确诊断出的短板。

文本渲染是我首先需要攻克的难题。GEdit-Bench 上 text_change 仅 1.58 的成绩, 暴露了当前模型在字符级控制上的根本不足。解决方案已然清晰: 引入一个与 Glyph-ByT5 字形编码器配合的专用“文字专家”。这个专家不需要去处理什么全局构图, 它唯一的任务就是在 Gate 给它路由来文字区域 token 时, 精准地画出每一个字符的笔画、字距和颜色。同理, 计数能力的缺失——DPG-Bench 上 Other-Count 仅 46.5——也需要这样专门的数值推理模块来补齐。

而在更宏观的架构层面, 编辑与生成的任务冲突是 Dense 模型绕不开的问题。同一个 FFN 在面对编辑任务时要做到精细的像素保留, 面对生成任务时又要完成创造性的内容合成, 这两个目标在参数层面天然存在内在张力。我的 MoE 框架为这个难题提供了一个优雅的出口: 让它们各自拥有独立的专家子网络。至少两个视觉路由专家, 一个偏向于结构保持和局部修改, 另一个偏向于自由合成和风格化创作, 再配上共享专家处理所有 token 共有的通用知识。这样一来, 模型就自动化地、在底层就实现了任务的分流, 而不需要增加任何额外的模型规模或改动结构。

9.2 中期: 跨模态统一

视觉内的统一只是第一步。真正的 AIGC 创作闭环, 需要打通音频这一关键模态。

当前工业界(如 Seedance 2.0、SkyReels-V4)的做法是“双分支”架构——前半段统一语义理解, 后半段拆成视频和音频两个独立的 DiT 分支, 最后再合成。这有效, 但它错过了“视听共生”的更大潜力。在我的 MoE 框架下, 这恰好可以做得更干净: 只需增加一个专门的“音频专家”, 然后把视频 VAE 的潜变量和音频 VAE 的潜变量在序列维度拼接, 作为一个统一的时空序列送入 DiT-MoE 主干。共享的自注意力层会天然学到音画之间的毫秒级双向互动——视频看到音频才知道手指该按在哪里, 音频看到视频才知道该发出什么音调。而那个 Gate, 它会在静音片段自动把音频专家的路由权重降到零, 在音画同步的关键帧把相关 token 精准发送过去。计算是稀疏的, 能力是统一的。

这不再是“先画画、后配音”的串行流程, 而是一个真正同步的、端到端的创作引擎。视频的每一帧和音频的每一毫秒, 都在同一个潜空间里被同时生成。

9.3 远期：世界模型与具身智能

当模型能够统一处理视觉和音频以后，再往前的每一步都会指向同一个终极目标：世界模型。

“编辑优先”框架的核心能力——“动作保持，结构保留”——与世界模型对时空一致性的根本要求不谋而合。要让它走向真实世界，就需要融入更多物理先验，比如刚体碰撞、流体运动、因果推理。这可能意味着引入专门的“物理专家”，或者更专门的 3D/4D 视觉表征模块。同样地，这些新的能力不需要推翻旧架构，只是向 MoE 块里增加一个新的路由专家。

这个进化路线的终点，是让模型从“生成看起来逼真的像素”，彻底转向“预测物理上可信的未来”。当它能理解物体的重力、惯性和碰撞，理解一个动作背后的因果链时，它就不再只是一个内容生成工具，而是一个用于模拟和推演未来的世界引擎。而“编辑优先”框架中本就蕴含的“指令到视频”能力，在融入这种物理级预测后，将直接赋能具身智能——让机器人通过“语言指令 目标状态预测 动作序列规划”的完整闭环，在真实世界中执行任务。

这是一条从“视觉统一”到“通用智能引擎”的路径。每一步都建立在我现在这个“编辑优先、资源高效”的概念验证基石之上。而 MoE 架构，会是我走完这整段旅程时，最值得信赖的同行者。

10 结论与展望

本文系统性地诊断了 Token-Choice 稀疏 MoE 在视频扩散 Transformer 上的训练失效模式。通过一系列对比实验，我们揭示了从全局软饱和到选择性锁死的层次化失效过程，发现了 bfloat16 精度陷阱对专家权重更新的关键影响，并基于 30 层网络中超过 6500 万 token 的路由决策时间序列数据，提出了功能冗余假说。

我们的诊断与 ProMoE(5) 关于视觉 token 空间冗余性的分析、以及 MoE 专家专业化迷思 (11) 关于路由反映几何结构的发现高度一致，从训练动力学角度补充了这些工作的理论框架。

这项工作代表了针对现实视频生成 MoE 部署场景中路由失效的首次全面经验性诊断之一。它明确了 Token-Choice 范式在当前条件下的能力边界，并规划了从多模态统一到世界模型的清晰进化路线。

在未来的工作中，我们计划沿着本研究标定的范式边界向外探索，具体包括：

1. 在共享专家能力成熟后释放路由专家进行全专家协同训练，验证功能冗余假说的核心预测——全专家训练是否能“唤醒”锁死层。
2. 探索 Expert-Choice 路由作为 Token-Choice 的替代范式，以打破专家选择的被动性。
3. 将任务类型嵌入整合至路由器输入，实现 9 合 1 任务的显式感知路由。
4. 在跨架构的视频扩散模型上验证锁死诊断的泛化性。
5. 研究更大规模的路由专家数量(4-n 个)下，选择性锁死的演变规律。

致谢

本研究完全由作者自筹资金完成。作者衷心感谢开源社区——从预训练模型、代码框架到评估基准，社区的无私分享使独立研究者能够触及多模态生成的前沿。特别感谢 Kiwi-Edit、Wan2.2、HuggingFace Diffusers 以及所有匿名审稿人的贡献和启发。

References

- [1] Shazeer, N., Mirhoseini, A., Maziarz, K., et al. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. ICLR, 2017.
- [2] Fedus, W., Zoph, B., Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. JMLR, 2022.
- [3] DeepSeek-AI. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. Technical Report, 2026.
- [4] Alibaba Cloud. Qwen3: Qwen3 Technical Report., 2025.

- [5] Wei, Y., Zhang, S., Yuan, H., et al. Routing matters in MoE: Scaling diffusion transformers with explicit routing guidance. ICLR, 2026.
- [6] Kuaishou Technology. DiffMoE: Mixture-of-experts for diffusion models. CVPR, 2025.
- [7] Shi, Y., et al. Mamoda2.5: Enhancing unified multimodal model with DiT-MoE. arXiv:2605.02641, 2026.
- [8] Jiang, K., et al. Mixture of distributions matters: Dynamic sparse attention for efficient video diffusion transformers. arXiv, 2026.
- [9] Song, Y., Sohl-Dickstein, J., Kingma, D.P., et al. Score-based generative modeling through stochastic differential equations. ICLR, 2021.
- [10] DREAM: Dynamic routing of experts via attention-based mixture for vision-language-action modeling. Knowledge-Based Systems, 2026. <https://www.ebiotrade.com/newsf/2026-2/20260227000821269.htm>
- [11] Dai, D., et al. The myth of expert specialization in MoEs: Why routing reflects geometry, not necessarily domain expertise. arXiv, 2026.
- [12] T. Lunyan. Analysis of Data Type Issue with TopKRouter Expert Bias in Megatron-LM. Git-Code Blog, May 2025. <https://blog.gitcode.com/fbce3dac83adb53fcf8c720b4b0c06dd.html>.
- [13] Y. Li. The Stability Gap: Why Top-K Routing Breaks RL Optimization. Personal Blog, December 2025. <https://richardli.xyz/post/topk-routing-stability-gap/>.
- [14] Load balancing mixture of experts with similarity preserving routers. arXiv, 2025.
- [15] Zhou, Y., et al. Mixture-of-Experts with Expert Choice Routing. arXiv, 2022.
- [16] Chen, J., et al. Sparse VideoGen: Accelerating video generation with sparse attention. arXiv, 2025.
- [17] Sha, H., Zheng, Y. UniGen-LingXi: A Resource-Efficient, Editing-First Framework for 9-in-1 Multi-Modal Generation and Editing. arXiv, 2026.
- [18] Fei, Z., Fan, M., Yu, C., Li, D., Huang, J. Scaling Diffusion Transformers to 16 Billion Parameters. arXiv preprint arXiv:2407.11633, 2024.